

Panduan Belajar: GitHub & Hosting

Tugas Besar Big Data · Credit-Card Default (Taiwan 2005)

Penjelasan lengkap — konsep dulu, baru perintah — untuk dipelajari.

Dokumen ini menjelaskan **dua hal**: (1) cara kerja **Git/GitHub** untuk mengirim kode, dan (2) cara **hosting** stack ini (Kafka + Spark + Hive) di server cloud gratis dengan aman. Setiap bagian dimulai dari **konsep** ("kenapa"), lalu **langkah** ("bagaimana"), lalu **troubleshooting**. Dikaitkan dengan file nyata di repo Anda.

BAGIAN 1 — GITHUB & GIT

1.1 Git vs GitHub (jangan tertukar)

- **Git** = program di komputer Anda yang merekam *riwayat perubahan* file (version control). Jalan offline.
- **GitHub** = layanan online tempat menyimpan salinan repository Git agar bisa di-backup/dibagikan.

Analogi: **Git** = mesin perekam riwayat, **GitHub** = Google Drive untuk hasil rekamannya.

1.2 Konsep inti

Istilah	Arti	Di proyek Anda
repository (repo)	folder proyek yang dilacak Git	Big Data_Tubes
commit	satu "snapshot" perubahan + pesan	mis. "tambah report.pdf"
branch	jalur versi; default <code>main</code>	pakai <code>main</code> saja
remote	alamat repo di server (GitHub)	dinamai <code>origin</code>
staging area	ruang tunggu sebelum commit (<code>git add</code>)	—
<code>.gitignore</code>	daftar file yang tidak dilacak	<code>.env</code> , <code>raw/*</code> , dll
clone	menyalin repo dari GitHub ke komputer	dipakai di VM saat hosting
push / pull	kirim / ambil commit ke/dari remote	—

1.3 Siklus kerja Git (wajib paham)

```
file berubah -> git add -> git commit -> git push (working dir) (staging) (riwayat lokal) (GitHub)
```

- `git add` = memilih perubahan yang akan disimpan (pindah ke *staging*).
- `git commit` = menyegel pilihan jadi titik riwayat permanen + pesan.
- `git push` = mengunggah riwayat lokal ke GitHub.

1.4 `.gitignore` & keamanan (KRUSIAL)

File `.gitignore` proyek Anda berisi antara lain:

```
.env # kunci FRED API ada di sini -> JANGAN ke GitHub raw/* # data mentah besar **/logs/* # log __pycache__/#
cache Python
```

Bahaya nyata: jika `.env` ter-push, kunci FRED API jadi publik dan bisa disalahgunakan. Karena `.env` sudah diabaikan, ini aman. Kunci asli di `.env.example` juga sudah diganti placeholder `your_fred_api_key_here`, jadi tidak ada rahasia di repo.

Aturan emas: rahasia (API key, password) **tidak pernah** masuk Git. Selalu lewat `.env` yang di-ignore + sediakan `.env.example` sebagai template kosong.

1.5 Kenapa artefak besar diabaikan

Pilihan Anda: abaikan `warehouse/staging/` (23 MB parquet) & `dashboard/exports/`, tetapi simpan `synthetic/synthetic_credit_clients.csv` (14 MB). Alasannya:

- `warehouse/staging/` adalah **hasil generate** -> bisa dibuat ulang dengan menjalankan pipeline.
- Git dirancang untuk **teks/kode**, bukan biner besar; parquet bikin riwayat berat.
- CSV sintetis adalah **input** pipeline (bukan output) -> disimpan agar reproduisibel tanpa menjalankan CTGAN yang berat.
- GitHub menolak file > 100 MB; semua file Anda di bawah itu.

Tambahkan ke `.gitignore`:

```
# generated warehouse / dashboard artifacts (regenerable) warehouse/staging/ dashboard/exports/
```

1.6 Autentikasi: kenapa password biasa tidak bisa

Sejak 2021 GitHub menolak login password lewat git. Pilihan:

1. **Personal Access Token (PAT)** — "password sekali pakai" berisi izin tertentu (scope repo).
2. **Git Credential Manager** — bawaan Git for Windows; push pertama membuka browser untuk login, lalu kredensial tersimpan. **Paling mudah.**
3. **SSH key** — pasangan kunci kriptografi; lebih teknis, cocok jangka panjang.

Untuk Anda: andalkan **Credential Manager** (jendela login otomatis). Jika diminta password manual, isi dengan **PAT** (bukan password akun). Membuat PAT: GitHub -> Settings -> Developer settings -> Personal access tokens -> Tokens (classic) -> Generate new token -> centang scope **repo** -> Generate -> salin.

1.7 Langkah lengkap push ke GitHub

```
# (sekali) identitas git config --global user.name "Nama Anda" git config --global user.email
"fajrihadiid@gmail.com" # 1. tambahkan warehouse/staging/ dan dashboard/exports/ ke .gitignore (lihat 1.5) #
2. inisialisasi cd "b:\Download\Big Data_Tubes" git init git branch -M main # 3. pilih & segel git add -A git
status # WAJIB cek: .env & warehouse/staging TIDAK boleh muncul git commit -m "Tugas Besar Big Data: ETL & ELT
+ warehouse + dashboard + docs" # 4. buat repo KOSONG di https://github.com/new (tanpa
README/license/.gitignore) # 5. hubungkan & kirim git remote add origin
https://github.com/USERNAME/NAMA-REPO.git git push -u origin main
```

Update berikutnya: `git add -A -> git commit -m "..."` -> `git push`.

1.8 Troubleshooting Git

Gejala	Penyebab	Solusi
<code>.env</code> muncul di <code>git status</code>	belum di-ignore / terlanjur ter-add	cek <code>.gitignore</code> ; jika terlanjur: <code>git rm --cached .env</code> lalu commit

failed to push ... rejected	remote punya commit yang belum Anda punya	<code>git pull --rebase origin main</code> lalu <code>git push</code>
diminta password berulang	Credential Manager tak aktif	pakai PAT sebagai password
push pertama lambat	wajar (kirim semua file)	tunggu; berikutnya hanya delta
salah remote URL	typo saat <code>remote add</code>	<code>git remote set-url origin <URL-benar></code>

BAGIAN 2 — HOSTING (Oracle Cloud, gratis, ARM)

2.1 Apa arti "hosting" di proyek ini

Hosting = menjalankan stack (Kafka + Spark + Hive) di **server yang menyala 24 jam dengan IP publik**, sehingga bisa diakses dari internet — bukan hanya di laptop.

Beda dengan GitHub:

- **GitHub** menyimpan **kode** (statis, untuk dibaca/clone).
- **Hosting** menjalankan **program** (hidup, melayani query/dashboard).

2.2 Kenapa Oracle Cloud "Always Free"

Stack butuh ~7,5 GB RAM. Free tier lain (AWS/GCP/Azure) hanya ~1 GB -> tidak muat. Oracle **Always Free Ampere A1 (ARM)** memberi **4 core + 24 GB RAM gratis selamanya** — satu-satunya yang cukup. **Konsekuensi:** prosesoranya **ARM (aarch64)**, beda dari laptop (x86/amd64), jadi image Docker harus punya versi ARM.

2.3 Konsep arsitektur CPU (kenapa image Spark diganti)

Program terkompilasi untuk arsitektur CPU tertentu. Hasil verifikasi:

- `apache/hive`, `apache/kafka`, `postgres`, `kafka-ui` -> **punya ARM** (tidak diubah)
- `bitnami/legacy/spark` -> **tidak punya ARM**

Maka di `docker/Dockerfile.spark` basis diganti ke `apache/spark:3.5.3-python3` (resmi, multi-arch, sudah termasuk Python/PySpark). Karena image `apache` tak punya "tombol" `SPARK_MODE` seperti `bitnami`, perintah `start master/worker` ditulis eksplisit di `docker-compose.yml`.

2.4 Konsep Docker & Docker Compose

- **Image** = cetakan/blueprint program (mis. image Hive).
- **Container** = image yang sedang berjalan (instance hidup).
- **Volume** = penyimpanan permanen container (data tak hilang saat restart) — mis. `warehouse`, `postgres-data`.
- **docker-compose.yml** = satu file mendefinisikan **banyak** container + jaringan + volume, dijalankan dengan satu perintah `docker compose up`.

Proyek Anda punya 7 layanan: `kafka`, `kafka-ui`, `spark-master`, `spark-worker`, `postgres`, `hive-metastore`, `hiveserver2`.

2.5 Model keamanan — DEFENSE IN DEPTH (paling penting dipelajari)

Layanan Anda (Hive, Spark, Kafka) **tanpa autentikasi & plaintext**. Jika dibuka langsung ke internet, siapa pun bisa menghapus tabel atau menjalankan kode. Solusi = **pertahanan berlapis**:

Lapis 1 — Port hanya di localhost.

Di docker-compose.yml semua port di-bind ke 127.0.0.1 (bukan 0.0.0.0). Artinya hanya bisa diakses dari dalam VM, tidak dari internet.

```
"127.0.0.1:10000:10000" <- aman: cuma localhost "10000:10000" <- BAHAYA: terbuka ke seluruh internet
```

Lapis 2 — Firewall hanya buka SSH (port 22).

Oracle punya 2 firewall (cloud "Security List" + host "ufw"). Keduanya hanya membuka port 22, dibatasi ke IP Anda.

Lapis 3 — Akses lewat SSH Tunnel.

Karena layanan terkunci di localhost VM, "salurkan" ke laptop lewat SSH terenkripsi:

```
ssh -N -L 10000:localhost:10000 ubuntu@<IP_PUBLIK_VM>
```

Arti -L 10000:localhost:10000: "port 10000 di laptop saya, sambungkan ke localhost:10000 di VM lewat terowongan SSH". Setelah ini dashboard konek ke localhost:10000 seakan Hive ada di laptop — padahal di cloud, dan lalu lintasnya terenkripsi + butuh kunci SSH.

Lapis 4 — Hardening VM: login SSH kunci-saja, fail2ban (blok brute-force), update otomatis, chmod 600 .env.

Kenapa tidak pasang password di Hive/Kafka langsung? Karena rumit (Kerberos/LDAP). SSH tunnel memberi enkripsi + autentikasi kuat untuk seluruh stack tanpa mengubah konfigurasi apa pun — ini praktik standar untuk infrastruktur data internal. Detail: docs/HOSTING_ORACLE.md.

2.6 Mode aman vs mode terbuka

	Mode AMAN (default)	Mode TERBUKA (insecure)
	Perdocker compose up -d	... -f docker-compose.yml -f docker-compose.prod.yml up -d
Port	hanya 127.0.0.1	publik 0.0.0.0
Akses	lewat SSH tunnel	langsung <IP>:10000
Risiko	rendah	tinggi (hanya untuk demo singkat)

File docker-compose.prod.yml sengaja dilabeli insecure dan hanya aktif bila Anda menambahkannya secara eksplisit.

2.7 Alur hosting end-to-end

```
1. Buat VM ARM 24GB di Oracle (console web) 2. Firewall: buka port 22 saja (2 lapis: VCN + ufw) 3. Hardening VM (SSH key-only, fail2ban, auto-update) 4. Install Docker di VM 5. git clone dari GitHub Anda (<- di sinilah Bagian 1 nyambung!) 6. Isi .env (FRED key) di VM 7. docker compose build && up (base file = mode aman) 8. Jalankan pipeline (isi warehouse Hive) 9. Buka SSH tunnel dari laptop 10. Tableau / Power BI -> localhost:10000 11. Selesai demo -> teardown (down -v + terminate VM)
```

2.8 Bagaimana GitHub & Hosting saling terhubung

- 1. Anda push kode ke GitHub (Bagian 1).
- 2. Di VM cloud, Anda clone dari GitHub itu (git clone https://github.com/.../repo.git).
- 3. VM menjalankan kode tersebut.

GitHub jadi **jembatan** memindahkan kode dari laptop ke server. (Alternatif tanpa GitHub: `scp/rsync` langsung, tapi lewat GitHub lebih rapi & ter-backup.)

2.9 Ringkas langkah kunci di VM (rujuk docs/HOSTING_ORACLE.md untuk lengkap)

```
# firewall host (hanya SSH) sudo ufw default deny incoming && sudo ufw default allow outgoing sudo ufw limit 22/tcp && sudo ufw --force enable # install docker (arm64) + compose plugin -> lihat HOSTING_ORACLE.md sec.3 #
ambil kode & jalankan git clone https://github.com/USERNAME/NAMA-REPO.git && cd NAMA-REPO cp .env.example .env
&& nano .env # isi FRED_API_KEY docker compose build docker compose up -d docker compose ps # semua Up #
jalankan pipeline (skip CTGAN; tahap tulis warehouse pakai -u root) docker compose exec spark-master python
/app/etl_pipeline/extract.py docker compose exec spark-master spark-submit /app/etl_pipeline/transform.py
docker compose exec -u root spark-master spark-submit /app/etl_pipeline/load.py docker compose exec -u root
spark-master spark-submit /app/elt_pipeline/extract_load.py docker compose exec -u root spark-master
spark-submit /app/elt_pipeline/run_transform.py docker compose exec -u root spark-master spark-submit
/app/dashboard/build_dashboard_kpis.py
```

Dari laptop, buka tunnel lalu konek BI ke `localhost:10000`.

2.10 Biaya & catatan realistis

- Oracle Always Free = **gratis selamanya** untuk shape Ampere terpilih, tetapi kapasitas ARM sering "Out of capacity" -> coba ulang / region lain.
- VM harus **menyala** agar bisa diakses; **terminate** setelah selesai agar tak ada layanan mengganggu terbuka.
- Untuk demo cepat, alternatif: jalankan di laptop + tunnel gratis (Cloudflare/ngrok). Tetapi Anda memilih jalur Oracle.

2.11 Troubleshooting hosting

Gejala	Penyebab	Solusi
Tidak bisa SSH ke VM	firewall VCN belum buka 22 / IP salah	cek Security List + IP publik
docker compose ditolak	user belum di grup docker	sudo usermod -aG docker \$USER lalu logout/login
Hive ter-OOM (exit 137)	RAM kurang saat CTGAN	jangan jalankan CTGAN bareng Hive; pakai CSV sintetis
BI tak konek via tunnel	tunnel belum jalan / DSN pakai IP	pastikan <code>ssh -L ...</code> aktif; DSN pakai <code>localhost</code>
build Spark ARM gagal	image tidak cocok	fallback QEMU (lihat HOSTING_ORACLE.md sec.9)

Saran urutan belajar

- Kuasai **siklus Git** (1.3) + `.gitignore` (1.4) — paling sering dipakai.
- Latih push satu repo kecil sampai lancar.
- Pahami **konsep Docker** (2.4) sebelum menyentuh cloud.
- Pahami **keamanan berlapis** (2.5) — nilai plus besar di laporan/penilaian.
- Praktik **Oracle VM** mengikuti `docs/HOSTING_ORACLE.md` langkah demi langkah.

Rujukan file di repo

- `docs/HOSTING_ORACLE.md / .pdf` — panduan hosting lengkap (bilingual).
- `docs/CARA_MENJALANKAN.md / .pdf` — cara menjalankan pipeline.
- `docker-compose.yml` — definisi stack (mode aman).
- `docker-compose.prod.yml` — override ekspos publik (insecure).
- `docker/Dockerfile.spark` — image Spark berbasis ARM.
- `.gitignore / .env.example` — kontrol rahasia & file diabaikan.